A bounded qualification of PrOntoQA-OOD

Finite generator capacity and citation-local scoring

SeyedAli Mirmostafa

Public technical note RC3 | Q01 | Frozen execution: 10 October 2026 | Retrospective audit: 11 October 2026

Abstract

We audit a stopped, locally frozen generation-and-scoring qualification at one pinned PrOntoQA-OOD revision. Each of two identical passes recorded 1,574 generator invocations and 44 valid returned examples. Relevant-distractor depths 5 and 7 returned none. Source analysis proves a capacity incompatibility for accepted depth-7 examples under the exact configuration: at least 15 initial property families would be necessary, but only 13 are provided. A structured citation-local scorer passed its registered fixtures and recorded finite-model cross-check. These are implementation-qualification findings; no LLM difficulty or recursive self-improvement was measured.

1. Research question and method

Can this exact generator configuration supply the required examples, and can a separately specified scorer distinguish unsupported justifications from valid alternative proofs, incomplete traces and formatting failures? The study uses main/PrOntoQA-OOD, not the original PrOntoQA v1 branch [1,2,4].

Q01 was frozen locally before execution (seal: 2026-10-10T20:14:27.4321447Z); this was not externally timestamped public preregistration. The revision was 0a6412b6fddf46324a1cb96e066dd7b3d89b87d6. Common options were fictional ontology, default vocabulary, random formula ordering, ModusPonens, proof_width=2, proofs_only=false, dfs_mode=none and no_adjectives=false. A fresh upstream namespace/configuration was created before every invocation. The harness statically loaded the audited generator definitions rather than running upstream model dispatch. The default ontology configuration has max_child_count=1, properties and negation enabled [4,E1].

The five cells were relevant distractors at depths 3/5/7 and no distractors at depths 5/7. Each used streams 101, 211, 307 and 401, with attempt seed base_seed*1000+attempt (one-based), applied to Python and NumPy. Each stream stopped at four returned examples or 100 invocations; no replacement streams or extensions were permitted. The registered replay repeated exactly the same protocol in a fresh process. The total cap was 4,000 invocations across both passes, with a 60-second limit per pass [E1].

2. Reconciled generation results

Distractors	Depth	Invocations	Returned / valid / unique formal keys	Streams with >=1 valid
relevant	3	265	16 / 16 / 16	4
relevant	5	400	0 / 0 / 0	0
relevant	7	400	0 / 0 / 0	0
none	5	232	16 / 16 / 16	4
none	7	277	12 / 12 / 12	4

Each pass contains 44 returns, 1,530 rejections and no recorded exceptions or returned-check errors. The replay contributes no new examples: there are 44 distinct formal theory/query keys, not 88. Combined invocations are 3,148. Internal runtimes were 1.469 and 1.672 seconds; both launches exited successfully within the cap [E2]. These stopped-stream totals are descriptive, not estimates of population generation probabilities.

All saved returned examples passed consistency, answer-label, reference-depth, minimum forward-MP depth and sound-complete structured-trace checks. Depths 3/5/7 correspond here to 2/4/6 MP inferences, because the initial fact counts as one step. At none-depth-7, streams 307 and 401 retained only 1 and 3 returns at their 100-attempt caps. "Successful stream" means at least one valid return, not reaching the four-return target. The none cells have only two coarse degree signatures each; these are not graph-isomorphism counts or evidence of LLM difficulty [E2].

The required relevant-depth-5 gate (>=8 unique valid examples across >=3 streams) failed. The scorer, artifact and replay checks passed, as did both none supply gates. The original STOP remains unchanged [E1,E2].

3. Configuration-specific depth-7 impossibility

Proposition. Under the pinned Q01 relevant/fictional/ModusPonens/width-2/default-vocabulary configuration, generate_question cannot return an accepted depth-7 example. This is a source-level constraint, not an inference from observing 0/400.

Proof. (i) Depth 7 includes the initial fact, so an accepted proof requires six MP inferences. The start-node filter requires main-tree level L>=6 (proof.py:54). (ii) Its root-to-start genealogy entails at least L+1>=7 recursive generate_ontology entries. Every entry with a family available removes one family before deciding whether to stop or create children (theory.py:75-97,158-159). If an entry instead encounters no families, the later spare-family guard already fails. (iii) Each of the six visited inference nodes must have a distractor parent: a node with only its true parent is rejected (proof.py:106-108). The zero-parent main root cannot supply an accepted MP inference: it has no assigned properties and type questions are disabled here (theory.py:15-24,163-169; run_experiment.py:543-546). The distractor parents require at least six distinct additional family removals (theory.py:114-127); sharing between siblings does not reduce consumption along one genealogy. (iv) Thus an admissible depth-7 path consumes at least 7+6=13 families before the auxiliary distractor ontology. The upstream guard requires at least two families still available (run_experiment.py:498-499). Acceptance therefore requires at least 15 initial families, but the fixed list contains 13 (356-368). Contradiction [4].

The counting statement concerns a path satisfying the per-hop distractor requirement. A level-6 node with missing distractor parents need not consume 13 families, but cannot yield the accepted proof. We do not claim the benchmark, other revisions, vocabularies, rules or distractor settings are impossible. No correction or enlarged inventory was tested. Depth 5 has the analogous necessary count 5+4+2=11<=13, so this argument does not prove its impossibility.

Relevant-depth-5 records contain 153 concept-exhaustion warnings, 224 property-exhaustion warnings and 23 silent records. Warnings do not identify the terminal rejecting guard; some returned cases elsewhere also have warnings. The cause of depth-5 rejection was not fully localized, and no post-run instrumentation or extra searches were performed [E2].

4. Exact scoring contract and qualification

The Q01 scorer is separate from the upstream analyzer. Its fragment is signed unary ground facts and universally quantified signed unary implications, interpreted classically; the original context must be consistent. For a fully parseable, citation-valid packet, let T_i contain only the explicitly cited context axioms and explicitly cited earlier sound claims. The endpoint is the earliest one-based i such that T_i does not entail claim_i. Global entailment under the original context is recorded separately. A globally true claim can therefore have a bad local justification; an unentailed claim need not be false [E1,E3].

4. Scoring contract and qualification (continued)

Valid alternative paths, contraposition and sound multi-hop shortcuts are accepted. No match to a reference proof is required. Bad, missing, future or duplicate citations are reference failures. Malformed or truncated JSON, unknown vocabulary or unsupported syntax are parsing failures. The whole packet and all citations are checked before inference scoring, so later format/reference failures suppress a logical-error index by design. Well-formed empty or partial traces can be incomplete without an unsupported claim. Final-answer correctness is separate. Whitespace and one optional terminal period are tolerated [E3].

Saved evidence: 25/25 fixtures match their specified expected fields (15 ok, 4 reference_error, 5 parse_error, 1 context_error). They cover alternative proofs, invalid inferences, true-but-badly-cited claims, incompleteness, incorrect answers and formatting. The recorded 2SAT/SCC versus exhaustive-assignment check reports 5,120 signed-query comparisons with zero mismatches: 16^2 ordered rule pairs * 5 zero/one-fact choices * 4 queries. Individual comparison rows were not saved; this retrospective audit reconciles the enumeration and recorded aggregate without rerunning it. The algorithms differ but share input normalization, so common encoding errors remain possible [E2,E3].

All 44 generated matched pairs preserve the entire trace/citations, axiom multiset and specified lexical/length/polarity features; context rule positions change. All 44 invalid members are globally true but locally badly justified. On a balanced set, any deterministic binary valid/invalid classifier restricted to the identical paired features has exactly 50% accuracy. This is not a measured first-error localization result. This is an analytic property, not a trained-baseline result. Position-sensitive citation matching and other shortcuts are not excluded [E2].

5. Interpretation, prior work and limitations

The defensible finding is a finite-resource incompatibility in one pinned implementation configuration, accompanied by a failed prespecified supply gate and a delimited scoring qualification. Generator rejection, scorer correctness on registered checks, and LLM difficulty are different questions. Q01 measured no model behavior, baseline headroom, statistical power, contamination resistance, causal feedback or RSI improvement. Replay demonstrates repeatability in the recorded environment, not independent cross-platform replication.

PrOntoQA already distinguishes valid reasoning from utility relative to a reference proof [1]. PrOntoQA-OOD already supports formal analysis, formatting variants, transitivity and modus tollens; its distractor construction uses fresh predicates at successive hops [2]. MATP also applies symbolic step checking to PrOntoQA-OOD, against full premises rather than Q01's explicit-citation endpoint [3]. We introduce no new theorem-proving method. The specific 13-family/spare-two incompatibility was not documented in the sources examined; this bounded review does not establish priority or novelty. An abstract generator permitting arbitrary depth does not guarantee every finite-vocabulary configuration can realize it [2,4].

The 44 examples are a small, stopped sample. Distinct formal keys are not independent structural families. The hand-written fixture set and two-predicate cross-check cannot certify unrestricted reasoning verification or natural-language parsing. No model was tested on the citation-local cases. Findings do not invalidate the published benchmark results or justify reopening the stopped RSI study.

6. Reproducibility and review

The accompanying public evidence archive preserves Q01 scientific records and the protocol/seal, source manifest/snapshot, harness/scorer, both journals/results, launches and original qualification report. Three historical files have local filesystem prefixes redacted; PROVENANCE.json links original and public hashes. Recorded original hashes are retained; MANIFEST.json hashes the public copies. It also retains the earlier preparation failure and feasibility/replay records as historical context; those are not pooled into Q01. Runtime: Python 3.12.14, NumPy 2.3.5, Windows. No packages were installed during Q01. Its cleared environment, audit restrictions and wall limit were defense in depth for inspected code, not an adversarial OS sandbox [E2,E3].

Private-original audit: all nine manifested source hashes, three harness hashes and the precommit hash match their saved values. Public qualification.py and oracle.py remain byte-identical; the redacted wrapper has a different public hash, linked in PROVENANCE.json and MANIFEST.json. Parsed pass cores match exactly; each journal matches its core.attempts. Timing, pass-number and outer replay fields are excluded. All ten bundled upstream files were separately compared byte-for-byte with the pinned official raw source; SOURCE_VERIFICATION.json records results. The seal predates both launches, but neither it nor the journals are externally tamper-proof. Precommit SHA-256:

`602f8516dba84f449d26cfaed6aeec24d0c6d08009db932a9b400d13df7bddb3`

Independent review status: separate Codex AI agents audited counts, source proof and closest literature. Two of those agents separately criticized the draft against the saved evidence and source; their required clarifications are incorporated. This is not independent human peer review; publication approval remains with the author. Review conclusions and limitations are included in the evidence archive. The release review is also AI-assisted, not peer review.

AI-assistance disclosure. Codex assisted the original Q01 protocol, harness/scorer, execution and interpretation, and prepared this retrospective note. Separate AI agents performed saved-record/source audits and draft criticism. Their algorithmic independence is limited by shared AI tooling, representation and context. This note and release preparation add document preparation, sanitization, licensing documentation, public-source comparison and record reconciliation only: no new generation, scorer execution, model calls, protected evaluation, credentials access or spending.

Recommendation: suitable as a modest public technical note, with this narrow scope and the reproducibility archive. It is an implementation qualification, not a novel RSI finding or evidence of LLM reasoning limitations. Reuse permissions were selected on 11 October 2026. RC3 corrects only the author-name spelling and release documentation; scientific results and the original STOP verdict remain unchanged.

Reuse permissions. Original Q01 code/scripts: Apache-2.0. This note and author-controlled documentation/records: CC BY 4.0. Upstream source retains Apache-2.0. Grants cover only rights the author can license; see LICENSING.txt and LICENSE_MAP.json. The scientific STOP remains unchanged.

References

[1] Saparov & He (ICLR 2023). Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought, Section 4. https://arxiv.org/html/2210.01240v4

[2] Saparov et al. (NeurIPS 2023). Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples, Section 3 and Appendices A.4/A.6/A.7. https://arxiv.org/html/2305.15269v3

[3] Beyond Correctness: Exposing LLM-generated Logical Flaws in Reasoning via Multi-step Automated Theorem Proving, Section 3.3. https://arxiv.org/html/2512.23511v1

[4] PrOntoQA-OOD pinned source. https://github.com/asaparov/prontoqa/tree/0a6412b6fddf46324a1cb96e066dd7b3d89b87d6 ; source locations are given in Section 3.

[E1] Frozen records: qualification_01/precommit.json; precommit_seal.json.

[E2] Saved outcomes: qualification_01/pass1_results.json; pass2_results.json; pass1_attempts.jsonl; pass2_attempts.jsonl; pass1_launch.json; pass2_launch.json.

[E3] Qualification code: qualification_01/qualification.py and oracle.py (unchanged); run_qualification.ps1 (public copy with executable path redacted). Source integrity: source_manifest.json. Original interpretation: qualification_01/qualification_report.txt.